

BSc (Hons) in Computer Science

Year 3, Semester 2

SE3092 : Platform Based Development

Assignment 01 – Technical Report

PulseFinance

Name	Registration number
M.P.H Perera	IT23596702
Sandali J.S	IT23651456
Shazaan AAM	IT23746596

November 2026

Contents

1 Introduction **1**

1.1 Purpose and scope . . . 1

1.2 How this report is organised . . . 1

2 Requirements Analysis **1**

2.1 Kavindu's situation . . . 2

2.2 Why four tools failed . . . 2

2.3 Problem statements . . . 3

2.4 Analytical lenses . . . 3

2.5 Functional requirements . . . 4

2.6 Non-functional requirements . . . 7

2.7 Constraints . . . 8

2.8 Out of scope . . . 8

2.9 Traceability: problems to requirements . . . 9

3 Architecture **9**

3.1 MVVM layers . . . 9

3.2 How data flows . . . 10

3.3 Dependency injection and platform integrations . . . 11

4 Firestore Schema **12**

4.1 Collections . . . 12

4.2 Why this structure . . . 13

4.3 Accounts . . . 13

4.4 Expenses . . . 14

4.5 Income . . . 14

4.6 Savings goal . . . 15

4.7 Multi-currency handling . . . 16

4.8 Maps on the user document . . . 16

4.9 Configuration: APIT tax bands . . . 16

4.10 Security rules . . . 17

4.11 Indexing strategy . . . 18

4.12 Local cache and entity-relationship model . . . 18

5 Annotated Wireframes **19**

5.1 Today (dashboard) . . . 20

5.2 Log an expense . . . 21
5.3 Income hub . . . 22
5.4 Quest (savings goal) . . . 23
5.5 Visual language . . . 24

6 Design Decisions and Justifications 24
DD-1 Offline-first: Room cache in front of Firestore . . . 24
DD-2 The received LKR decides; the exchange rate is frozen per entry (ADR-0001) . . . 25
DD-3 An Income Basis instead of a monthly budget . . . 25
DD-4 Unpaid invoices are tracked but never spendable; "overdue" is derived . . . 25
DD-5 The category proposes committed or discretionary; he decides . . . 25
DD-6 Bank SMS through the share sheet, not READ_SMS . . . 26
DD-7 A Spending Freeze is a self-commitment, and it never blocks . . . 26
DD-8 Subscriptions are detected from his own charges and one question . . . 26
DD-9 Tax bands are data in Firestore . . . 26
DD-10 A USD-linked goal scales the target he entered . . . 27
DD-11 Seasonal Peaks from the calendar and from his own history . . . 27
DD-12 Gaps are missing data, never failure . . . 27
DD-13 Amber instead of red, and one number first . . . 27
DD-14 Sensitive numbers behind biometrics; no card data at all . . . 28
DD-15 All calculations in domain use cases with unit tests . . . 28
DD-16 One quantified goal . . . 28

7 Implementation Status, Testing and Delivery 28
7.1 Requirement status . . . 28
7.2 Testing . . . 29
7.3 Project management . . . 29
7.4 Delivery . . . 30
7.5 Known limitations . . . 31
7.6 Team contributions . . . 31

8 Conclusion 31

1 Introduction

1.1 Purpose and scope

PulseFinance is a native Android application that gives one person an honest monthly picture of three things:

- **income** from four irregular, multi-currency sources;
- **spending**, split by whether he had to make it (committed) or chose to (discretionary);
- **progress towards one quantified goal**: the MacBook Pro M4.

1.2 How this report is organised

Section	Content
2 Requirements analysis	Scenario analysis, failed-tool analysis, problem statements, 73 requirements with status, constraints, traceability
3 Architecture	Layers, the MVVM diagram, offline-first data flow, dependency injection
4 Firestore schema	Collections, fields with Kavindu's data as examples, rules, local cache, ER model
5 Annotated wireframes	Four screens designed in Figma, each element annotated with the problem it answers
6 Design decisions	Sixteen decisions with context, alternatives and consequences
7 Implementation status	Testing, delivery, open gaps, team contributions

Requirement identifiers (**FR-INC-6**, **NFR-2**, ...) and problem identifiers (**P1**–**P11**) are used throughout so that any claim can be traced to the requirements in Section 2 and to the source code. The full Software Requirements Specification is in the repository at `docs/SRS.md`.

2 Requirements Analysis

2.1 Kavindu's situation

Aspect	What the scenario tells us
Who	25, junior software engineer in Colombo; comfortable with technology, low tolerance for friction in daily data entry.
Income	Salary (credited on the 25th); freelance web projects paid in milestones; Google AdSense in USD, credited in LKR at the bank's rate; crypto trading with a significant Q1 gain and a material Q3 loss. His own estimate of monthly income, "LKR 160,000–210,000", has never been validated.
Spending	Card, cash, PickMe, UberEats; rent LKR 34,000; a gym membership and two subscriptions auto-debited from a secondary account he rarely checks. Discovering "LKR 24,000 on coffee" was "genuinely shocking".
Savings	More than LKR 1.6 million earned in twelve months; LKR 11,200 saved.
Goal	MacBook Pro M4 from an authorised reseller, about LKR 490,000, within 12 months. "Emotionally present and financially invisible."
Behaviour	Optimistic about income, impulsive with discretionary spending, avoids looking at money when worried, and abandons tools once their numbers stop matching reality.

2.2 Why four tools failed

ID	Tool	Why it failed	What PulseFinance does instead
F1	Google Sheet "MacBook Fund"	Formulas broke when rows were inserted; entry was slow; after a three-week gap the incomplete rows felt discouraging.	All calculation lives in tested domain use cases, not editable formulas. Days without records are missing data, never zero or failure (**NFR-2**).
F2	Play Store budget app	USD only; no cloud sync; assumed one income source and a fixed monthly budget.	Income Source is a first-class concept; each entry keeps its original currency and the rate actually received; the allowance is recalculated daily (**FR-INC-1**, **FR-DASH-1**, **FR-AUTH-3**).
F3	Notes app	Pasted figures became an unreadable block after four days.	Every record is captured with structure – category, committed or discretionary, account – at entry time, through one-tap presets (**FR-EXP-1**, **FR-EXP-3**).
F4	Bank's spending summary	Card only; missed cash and the secondary account; filed a PickMe fare as "online retail".	Accounts include cash, the secondary account and the crypto exchange; the category only *suggests* committed or discretionary and Kavindu decides (**FR-ACC-1**, **FR-EXP-2**).

The common root cause is that each tool was **too fragile, too manual, too narrow or too unpleasant to open**. Each of those maps to a non-functional requirement: fragile to **NFR-6**, manual to **NFR-1**, narrow to the **FR-INC** group, unpleasant to **NFR-2** and **NFR-8**.

2.3 Problem statements

ID	Problem
P1	He cannot state his real monthly income; the estimate "LKR 160,000–210,000" was never validated.
P2	Freelance invoices are forgotten and payments cannot be matched to projects.
P3	AdSense income in USD is never recorded at the amount actually received.
P4	He has no running net crypto position and cannot answer "what did crypto earn me this year?".
P5	Spending is spread across card, cash, PickMe, UberEats and a secondary auto-debit account, with no single view.
P6	Two subscriptions he meant to cancel six months ago are still being charged.
P7	He has no idea of his committed versus discretionary split and has never seen how his spending is distributed.
P8	He does not know the monthly saving needed for LKR 490,000 in 12 months, because he does not know what he currently saves.
P9	Gaps in tracking made his previous tool feel discouraging and led to abandonment.
P10	He does not examine his payslip and cannot recall the amount deposited.
P11	He is impulsive with discretionary spending and has no friction at the point of purchase.

2.4 Analytical lenses

What must the system understand about his income? A single "income" number hides four things: the *source* (salary, freelance, AdSense, crypto); whether the money is *realised or only invoiced* – a pending invoice is not spendable; the *original currency and the rate actually received* – AdSense is credited at the bank's rate, not the market rate; and the *sign* – a crypto loss is negative income. Timing matters too: because the salary lands on the 25th, "income so far this month" is misleading for most of the month. The daily allowance is therefore based on an **Income Basis**: the higher of this month's realised income and the average of the previous three months, with crypto *profits* excluded from that average so that one lucky trade cannot inflate the allowance of an income-optimistic, impulsive spender (**FR-DASH-3**).

What must the system understand about his spending? Whether each expense was *committed* (rent, utilities, subscriptions) or *discretionary* (coffee, PickMe, dining), which *account* it came from, and whether a recurring charge is something he still uses. The bank summary failed because it guessed the category and saw only one card; PulseFinance proposes, he decides, and every account he uses is in scope.

What feedback makes a goal feel real? A goal becomes actionable when it is expressed in the same units as his daily choices: a required daily pace (about LKR 1,330 per day), a comparison with what he actually saves, an arrival date, and the cost of a habit in "days of MacBook". Progress must only ever move forward – no streaks to break.

2.5 Functional requirements

Priorities use MoSCoW (M, S, C). Status reflects the code on `main` at the time of writing.

ID	Requirement	Pri	Status
Identity and onboarding			
FR-AUTH-1	Register and sign in with email and password (Firebase Authentication).	M	Done
FR-AUTH-2	Sign in with a Google account.	S	Done
FR-AUTH-3	Every record stored under the signed-in user's ID, so the same data appears on any device.	M	Done
FR-AUTH-4	First-run setup captures salary, active Income Sources and fixed monthly commitments (e.g. rent LKR 34,000) in three steps.	M	Done
FR-AUTH-5	Setup records each source's expected variability (steady, varies, highly volatile).	C	Done
FR-AUTH-6	Setup lets him set an emergency-runway buffer of 1–6 months of committed costs.	C	Planned
FR-AUTH-7	A debug-only demo drawer fills credentials for a pre-seeded account; credentials come from build configuration.	M	Done
Income			
FR-INC-1	Each entry records source, title, client or platform, original amount, currency (LKR, USD, USDT, ETH) and date.	M	Done
FR-INC-2	The exchange rate used is stored with the entry and never recomputed (ADR-0001).	M	Done
FR-INC-3	Freelance income can be a Pending Invoice with client and due date, marked received in one tap.	M	Done
FR-INC-4	A pending invoice past its due date is automatically treated as overdue.	M	Done
FR-INC-5	The form pre-fills the day's suggested rate (open.er-api.com, CoinGecko), cached daily, usable offline with its age shown.	M	Done
FR-INC-6	For a foreign-currency entry he can type the LKR his bank actually credited; the effective rate is derived and stored.	M	Done
FR-INC-7	A crypto trade is a Crypto Trading Result: positive for a profit, negative for a loss.	M	Done
FR-INC-8	The Crypto year-to-date net is shown.	M	Done
FR-INC-9	Realised income and pending/overdue amounts are shown separately for the month, by source.	M	Done
Expenses			
FR-EXP-1	Each expense records amount (LKR), category, committed/discretionary flag, account, time and note.	M	Done

ID	Requirement	Pri	Status
FR-EXP-2	The category proposes committed or discretionary; he can flip it in one tap.	M	Done
FR-EXP-3	One-tap Sri Lankan presets (coffee, PickMe, UberEats, rent).	M	Done
FR-EXP-4	Presets include Dialog and Mobitel reloads and data packages.	S	Done
FR-EXP-5	Expenses can be attributed to any account, including cash and the secondary auto-debit account.	M	Done
FR-EXP-6	An expense saved offline appears immediately and syncs on reconnection.	M	Done
FR-EXP-7	A bank SMS shared through the share sheet becomes a draft expense (amount, merchant, card last four digits), without READ_SMS.	S	Done
Subscriptions			
FR-SUB-1	Recurring subscriptions are detected by grouping subscription-category expenses by merchant over 120 days.	M	Done
FR-SUB-2	Each shows its monthly and annual cost and the account it is charged to.	M	Done
FR-SUB-3	He records Keep or Cancel; a charge after Cancel flags it as “still charging”.	M	Done
FR-SUB-4	After a charge: “Did you use this this month?”; two consecutive Nos set the risk to High.	S	Done
FR-SUB-5	For app-linked subscriptions (Netflix, Spotify), usage statistics show when the app was last opened and feed the risk.	S	Partial
FR-SUB-6	Results are grouped by account, with the secondary account highlighted.	S	Done
Savings goal			
FR-GOAL-1	Set a goal with title, LKR target and date (for Kavindu: MacBook Pro M4, LKR 490,000, 12 months).	M	Done
FR-GOAL-2	The required monthly and daily saving is calculated.	M	Done
FR-GOAL-3	The actual savings rate is calculated: realised income minus spending for each 30-day block of the last 90 days, averaged over the blocks that have records, so a logging gap is never counted as zero saving.	M	Done
FR-GOAL-4	The arrival date is projected at the actual rate, stating whether he is on pace or behind, and by how much.	M	Done
FR-GOAL-5	Any amount can be translated into “days of goal funded” (opportunity cost).	S	Done
FR-GOAL-6	An optional USD reference price makes the LKR target follow the daily rate; the change is shown.	S	Done
FR-GOAL-7	Quick-add boosters (+LKR 5,000 / 10,000 / 25,000) move a windfall into the goal.	C	Done
Dashboard			

ID	Requirement	Pri	Status
FR-DASH-1	Safe-to-Spend Allowance per day for the rest of the month.	M	Done
FR-DASH-2	Pending and overdue invoices are never counted as spendable.	M	Done
FR-DASH-3	The three-month average in the Income Basis excludes crypto profits but keeps crypto losses.	M	Done
FR-DASH-4	The month's committed versus discretionary split as a bar with amounts.	M	Done
FR-DASH-5	An overdue-invoice banner with client and amount and one-tap "mark received".	M	Done
FR-DASH-6	Figures update in real time from Firestore listeners.	M	Done
Analytics and data			
FR-ANA-1	Category donut for the period, discretionary distinguishable from committed.	M	Done
FR-ANA-2	Week and month periods.	M	Done
FR-ANA-3	Daily spending bars with the peak day highlighted.	S	Done
FR-ANA-4	Day and year periods.	C	Done
FR-ANA-5	Transactions searchable by keyword and filterable by account, category and date.	C	Done
FR-DATA-1	Export all income, expenses and goals to CSV.	S	Done
Accounts and Spending Freeze			
FR-ACC-1	Accounts of type Primary Bank, Secondary Bank (autodebit), Cash and Crypto Exchange.	M	Done
FR-ACC-2	Combined balance across his recorded accounts.	S	Done
FR-ACC-3	A Spending Freeze on an account until a chosen date; the account shows as locked.	S	Done
FR-ACC-4	Spending from a frozen account shows the cost in goal days and offers "Log anyway"; recording is never refused.	S	Done
FR-ACC-5	Freeze Breaks are counted and shown.	S	Done
FR-ACC-6	Account numbers masked (• • • • 4521), revealed only after biometric or device-PIN authentication.	S	Done
FR-ACC-7	A revealed number can be copied for a freelance invoice; the clipboard is cleared after 60 seconds.	S	Done
Sri Lankan context			
FR-LK-1	Payslip: gross salary to employee EPF (8%), APIT and take-home; all editable.	M	Done
FR-LK-2	APIT tax bands read from Firestore configuration, not compiled into the app.	M	Done
FR-LK-3	Accumulating EPF balance (employee 8% + employer 12%) shown as savings he owns.	S	Done
FR-LK-4	Seasonal Peaks (Avurudu, Vesak, Poson, December) plus, after 12 months, his own months $\geq$20% above average.	S	Done
FR-LK-5	30 days before a peak: a warning with last year's excess and a one-tap "set aside".	S	Done

ID	Requirement	Pri	Status
FR-LK-6	Amounts in LKR with separators; foreign amounts show the original currency beside the LKR value.	M	Done

2.6 Non-functional requirements

ID	Quality	Requirement	Measurable target	Status
NFR-1	Capture speed	Logging a preset expense is fast enough to do at the point of purchase.	$\leq$3 taps, $\leq$5 s	Done
NFR-2	Gap tolerance	Logging gaps are never shown as zero spending, broken streaks or failure; averages only over periods with data.	No penalty, streak reset or "0 spent" for unrecorded days	Partial
NFR-3	Offline	All screens work offline from cached data; offline writes sync on reconnection.	Sync $\leq$10 s after reconnection	Done
NFR-4	Real-time	Changes from another device appear without a refresh.	$\leq$2 s online	Done
NFR-5	Data isolation	Each user can access only `users/{uid}`, enforced by Firestore rules.	No cross-user access	Done
NFR-6	Financial accuracy	Original amount, currency and rate kept per income entry; totals derivable from stored records.	Totals unchanged after a rate change	Done
NFR-7	Compatibility	Installs and runs on Android 8.0+, starts quickly.	minSdk 26, targetSdk 34	Done
NFR-8	Accessibility	Status never by colour alone; text usable at large font sizes.	Icon or label per status; 200% font scale	Planned
NFR-9	Robustness	No screen crashes or appears blank with no data, partial data or a network error.	Designed empty state everywhere	Done
NFR-10	Sensitive display	Account numbers masked by default, revealed only after a biometric check.	Clipboard cleared $\leq$60 s	Done
NFR-11	Portability	Data exportable in an open format.	CSV opens in Sheets/Excel	Done
NFR-12	Privacy	Minimum permissions; optional ones requested in context with an explanation.	No READ_SMS; usage access in context	Partial

2.7 Constraints

ID	Constraint
C-1–C-7	100% Kotlin; Jetpack Compose with Material 3; MVVM with no business logic in Composables; single activity with Navigation Compose; state as `StateFlow`; coroutines in `viewModelScope`; Hilt for dependency injection.
C-8, C-9	Firebase Authentication for identity; Cloud Firestore as the primary persistence layer, with `firestore.rules` submitted.
C-10, C-11	minSdk 26, targetSdk $\geq$34; the submitted APK connects to a live Firebase project.
C-12, C-13	No bank connection. Every balance and transaction is Kavindu's own record; "freeze" is a self-commitment, not a card block.
C-14	Card CVVs are never stored, displayed or requested.
C-15	READ_SMS is not used; bank SMS enter only through the share sheet.
C-16	Tax rules (APIT bands, EPF rates) are data, not code.
C-17	Exchange-rate services are free third parties: called at most once a day, and the app keeps working without them.
C-18	Demo credentials only in debug builds, through build configuration.

2.8 Out of scope

Item	Reason
Bank-feed import / open banking	Excluded by the brief (C-12); no Sri Lankan open-banking API available.
Real card blocking, CVV display	No bank integration (C-13); CVV storage forbidden (C-14).
Live crypto portfolio valuation	His question is realised year-to-date earnings (**P4**); live prices add noise, not clarity.
Several goals competing for money	One quantified goal is the scenario's focus; the model allows more goals but no allocation logic between them.
Geofenced gym-usage detection	Battery and privacy cost far outweigh the benefit; the charge-day question (**FR-SUB-4**) covers the need.
Sinhala/Tamil UI, "lakhs" formatting	Kavindu works in English and reads LKR in thousands; deferred.

2.9 Traceability: problems to requirements

Problem	Requirements that address it
P1 Unknown real income	FR-INC-1, FR-INC-9, FR-DASH-1, FR-LK-1, FR-AUTH-4
P2 Forgotten invoices	FR-INC-3, FR-INC-4, FR-DASH-5, FR-ANA-5, FR-ACC-7
P3 AdSense not recorded as received	FR-INC-2, FR-INC-5, FR-INC-6, NFR-6
P4 No net crypto position	FR-INC-7, FR-INC-8, FR-DASH-3
P5 Fragmented spending channels	FR-EXP-5, FR-EXP-7, FR-ACC-1, FR-ACC-2
P6 Forgotten subscriptions	FR-SUB-1 to FR-SUB-6
P7 No committed/discretionary view	FR-EXP-2, FR-DASH-4, FR-ANA-1, FR-ANA-2
P8 Unknown required saving rate	FR-GOAL-2, FR-GOAL-3, FR-GOAL-4, FR-GOAL-6
P9 Gaps cause abandonment	NFR-1, NFR-2, NFR-9, FR-DATA-1
P10 Payslip not understood	FR-LK-1, FR-LK-2, FR-LK-3
P11 Impulsive spending	FR-ACC-3 to FR-ACC-5, FR-GOAL-5, FR-LK-4, FR-LK-5

3 Architecture

3.1 MVVM layers

PulseFinance is a single-activity Compose app following MVVM with a domain layer (C-3). Dependencies point inwards: screens depend on ViewModels, ViewModels on use cases and repository interfaces, and the data layer implements those interfaces. Figure 1 is drawn from the constructors of the ViewModels on `main`. Each row reads left to right: a Compose screen, its ViewModel, the use cases and repository interfaces that ViewModel depends on, then the nine repository implementations that every row converges on, and finally the data sources only the data layer touches. Screens never reach a repository or data source directly, and only the repository implementations know about Room and Firestore. Every ViewModel also injects `AuthRepository` to obtain the signed-in user's ID; that link is omitted for readability. Dotted arrows are platform helpers rather than repositories.

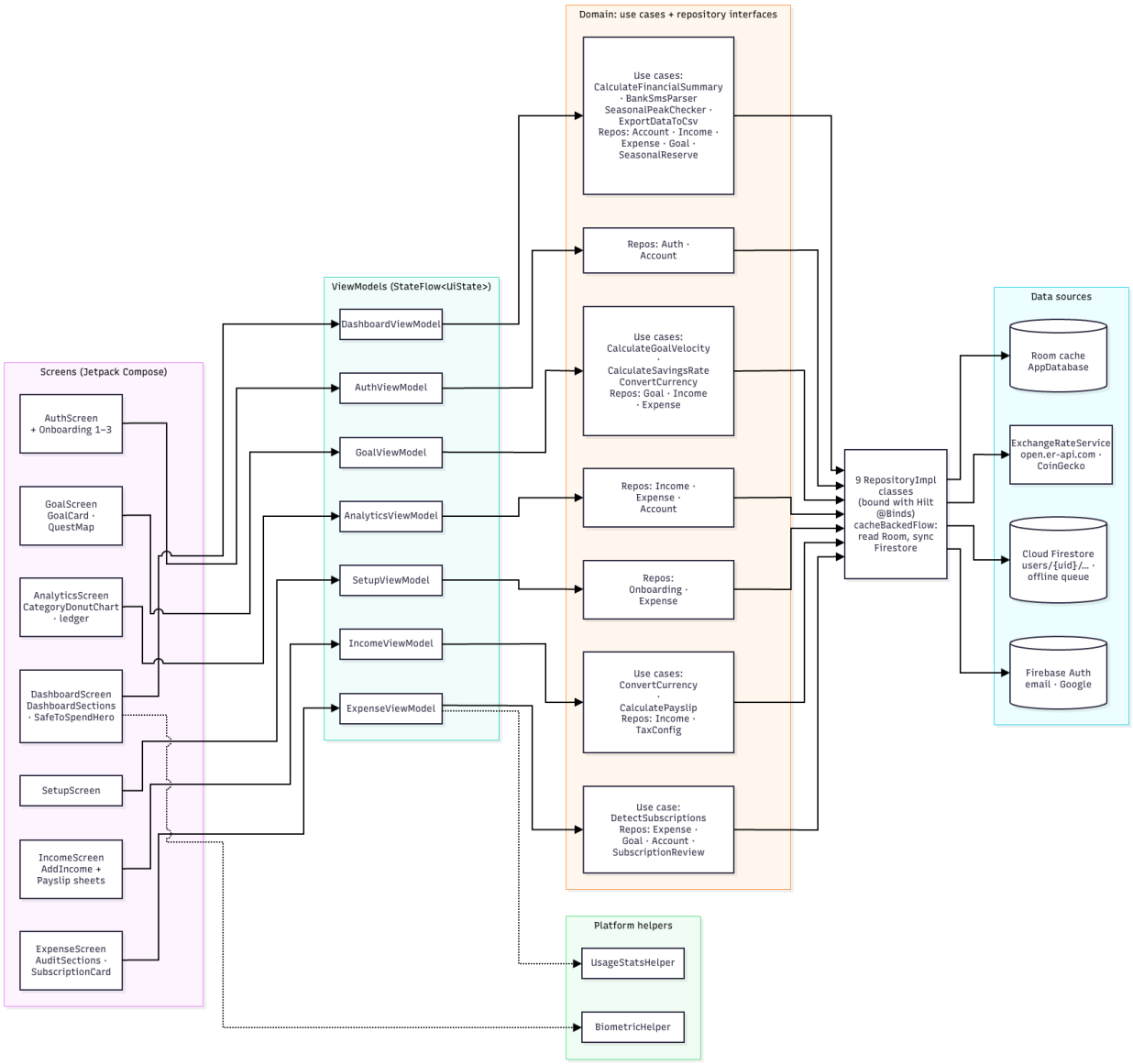


Figure 1: MVVM diagram. Screens (Jetpack Compose) → ViewModels exposing `StateFlow<UiState>` → domain use cases and repository interfaces → the nine `RepositoryImpl` classes, bound with Hilt `@Binds`, which read Room and sync Firestore through `cacheBackedFlow` → data sources. Dotted arrows lead to platform helpers.

3.2 How data flows

Reads. The account, expense, income and goal repositories return a `cacheBackedFlow`. The UI collects a Room `Flow`, so a screen shows cached data immediately with or without signal. While someone is collecting, a Firestore snapshot listener runs alongside it and writes every snapshot into Room with `replaceAll` in a transaction; Room then re-emits. When the screen goes away for more than five seconds (`WhileSubscribed(5000)`), the listener is removed. The goal repository also combines the goal with the day's USD rate, so a USD-linked target re-prices when a new rate arrives (**FR-GOAL-6**). Small per-user settings – subscription decisions, seasonal reserves, the onboarding profile, and the tax bands – rely on Firestore's own offline cache instead of Room.

Writes. Figure 2 shows Kavindu logging a coffee with no signal. Writes do not wait for the server: awaiting the Firestore task would hang until the device reconnects. Firestore's persistent queue, not an application flag, guarantees the upload.

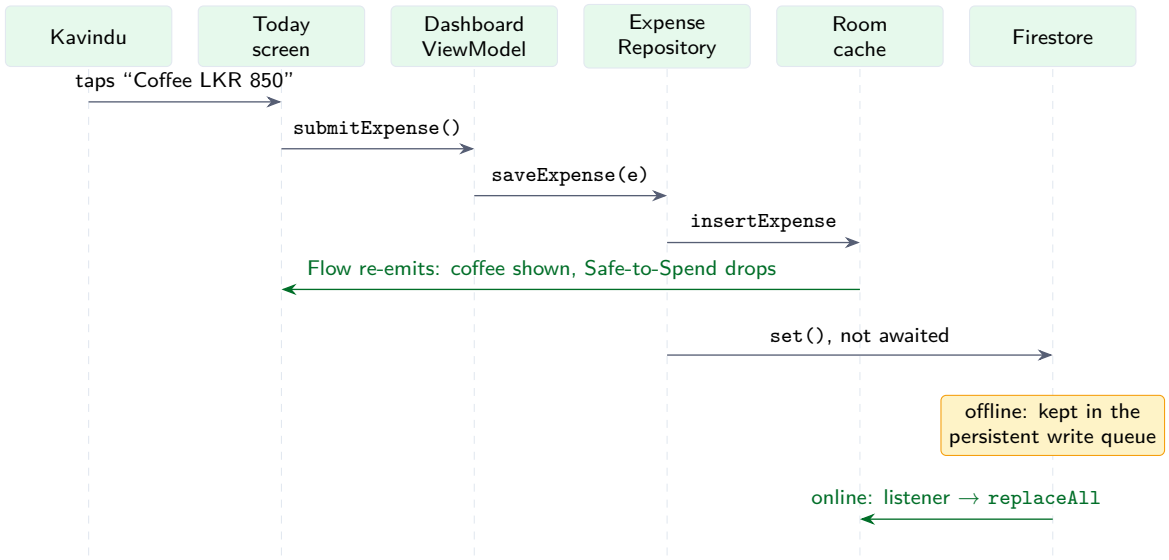


Figure 2: Recording an expense offline. The coffee is visible instantly (**NFR-1**, **FR-EXP-6**) and uploads on reconnection (**NFR-3**).

3.3 Dependency injection and platform integrations

Hilt provides the Room database and DAOs (`DatabaseModule`), Firebase Auth and Firestore with persistent cache settings (`FirebaseModule`), and binds all nine repository interfaces to their implementations (`RepositoryModule`). ViewModels are obtained with `hiltViewModel()` inside `AppNavGraph`; no ViewModel is created manually (C-7).

No business logic in Composables. Screens collect `uiState`, call ViewModel functions and lay things out; nothing else. Rules live in the domain and are unit-tested: for example, how many days an invoice is overdue is `IncomeStream.daysOverdue()`, and grouping subscriptions by account (**FR-SUB-6**) is `DetectSubscriptionsUseCase.groupByAccount()`. The domain layer has no Android imports. Large screens are split into section composables (`DashboardSections.kt`; `AuditSections.kt` and `SubscriptionCard.kt`), and every colour comes from `MaterialTheme.colorScheme` (Section 5).

Feature	Where	Android mechanism
Share a bank SMS (**FR-EXP-7**)	`MainActivity`, `BankSmsParser`	Share sheet (`ACTION_SEND`, text/plain); no READ_SMS
Reveal and copy account number (**FR-ACC-6**, **FR-ACC-7**)	`BiometricHelper`	`BiometricPrompt` (strong biometric or device credential); clipboard cleared after 60 s
Screens hidden from screenshots	`MainActivity`	`FLAG_SECURE`
App last opened (**FR-SUB-5**)	`UsageStatsHelper`	`UsageStatsManager` (special access)
CSV export (**FR-DATA-1**)	`ExportDataToCsvUseCase`, `CsvShare.kt`	The use case only builds the CSV text (plain Kotlin); the screen opens the share sheet (`ACTION_SEND`, text/csv)
Daily exchange rates (**FR-INC-5**)	`ExchangeRateService`	HTTPS on `Dispatchers.IO`, at most once a day (C-17)

4 Firestore Schema

4.1 Collections

Everything Kavindu records lives under his own user document, so a single rule protects it all (**NFR-5**) and the same data appears on any device he signs in on (**FR-AUTH-3**). App-wide configuration that is not his data lives in a separate read-only `config` collection.

Live database (Firebase console, project members only):

https://console.firebase.google.com/project/finance-app-se3092/firestore/databases/-default-/data

```
users/
  {uid}                                  user document
    subscriptionReviews : map            Keep / Cancel decisions            (FR-SUB-3, FR-SUB
    -4)
    seasonalReserves    : map            money set aside before a peak      (FR-LK-5)
    onboarding          : map            first-run setup answers            (FR-AUTH-4, FR-AUTH
    -5)
    accounts/{accountId}        AccountDto   where money is held or paid from  (P5)
    expenses/{expenseId}        ExpenseDto   money out                         (P5, P6, P7)
    income_streams/{incomeId}   IncomeDto    money in, received or invoiced    (P1-P4)
    goals/{goalId}              GoalDto      the savings goal (his MacBook)    (P8)

config/                                  read-only for signed-in users
  apitBands                     ApitConfigDto   APIT tax schedule              (FR-LK-2, C-16)
```

The three maps on the user document are small, are always read together with the user, and are written with `SetOptions.merge()` so that they never overwrite one another. Keeping them as maps rather than subcollections avoids three extra listeners per screen.

4.2 Why this structure

Decision	Why, for Kavindu's data
Everything he records lives under `users/{uid}`	Ownership is the path itself, so the security rule is one comparison and no query needs a `where userId == ...` filter. His records cannot mix with another user's, and deleting his account is deleting one subtree (**NFR-5**).
One collection per kind of record, not one mixed "transactions" collection	Income carries a currency, rate and invoice status (**P1**–**P4**); an expense carries a category and committed flag (**P7**). Separate collections let the rules validate each shape, and let each screen listen only to what it shows: the Income hub never downloads his expenses.
A document per record, not an array in one document	He logs a coffee or a PickMe most days, so a year is hundreds of expenses. One large document would approach Firestore's 1 MiB limit and every new expense would rewrite the whole list; a document per record keeps writes small and lets each offline write queue on its own (**FR-EXP-6**).
Small settings as map fields on the user document	Reviews, reserves and onboarding are a few entries each, read with the user and merged on write, so they need no extra listener.
`config` at the top level	The APIT bands are the same for every user and read-only for the app (**FR-LK-2**).
`userId` also stored in every record	Redundant with the path, but the rules check that the two match, so a record can never claim another user.
`paymentAccountName` copied onto each expense	Firestore has no joins; the ledger and the subscription audit (**FR-SUB-6**) show where money went without loading accounts. Accepted cost: renaming an account does not rename its old expenses.
Money stored in LKR, with income also keeping its original amount, currency and rate	Every total is a plain sum, and a later rate change never moves last month's numbers (**NFR-6**, ADR-0001).

4.3 Accounts

Path: `users/{uid}/accounts/{accountId}`.

On first sign-in, `ensureDefaultAccounts` creates one zero-balance account per place Kavindu's money leaves from, so every expense can be attributed from the first entry. Nothing else is invented: account numbers start empty. It checks the server first, so an empty offline cache is never mistaken for a new user.

Field	Type	Kavindu example	Why it exists
`id`	string	`acc_secondary_bank`	Fixed IDs for the four defaults
`name`	string	Secondary Account (Auto-Debits)	
`type`	string	`BANK_SECONDARY`	Primary, secondary, cash or crypto exchange (**FR-ACC-1**)
`balanceLkr`	number	0	His own record, never imported (C-13)
`isAutoDebitSource`	boolean	true	Where the gym and forgotten subscriptions are charged (**P6**)
`accountNumber`	string	empty	When set, shown masked until biometric reveal (**FR-ACC-6**)
`isFrozen`, `frozenUntilMillis`	boolean, number	false, null	Spending Freeze (**FR-ACC-3**)
`freezeBreakCount`	number	0	Times he spent from a frozen account (**FR-ACC-5**)

4.4 Expenses

Path: `users/{uid}/expenses/{expenseId}`.

Field	Type	Kavindu example	Why it exists
`title`	string	PickMe to office	Also the merchant key for subscription detection
`amountLkr`	number	1450	Always LKR
`category`	string	`TRANSPORT`	One of eight categories
`isCommitted`	boolean	false	Proposed from the category, flipped by him (**FR-EXP-2**, **P7**)
`paymentAccountId`, `paymentAccountName`	string	`acc_cash`, Cash in Hand	Covers cash and the secondary account (**FR-EXP-5**); name copied so lists render without a join
`timestamp`, `notes`	number, string		

4.5 Income

Path: `users/{uid}/income_streams/{incomeId}`.

Field	Type	Kavindu example (AdSense)	Why it exists
`source`	string	`ADSENSE`	Salary, freelance, AdSense or crypto (**P1**)
`title`, `clientOrPlatform`	string	AdSense – October; Google AdSense	For freelance, the client he invoices (**P2**)
`originalAmount`	number	45	In the original currency; negative for a crypto loss (**FR-INC-7**)
`currency`	string	`USD`	LKR, USD, USDT or ETH
`exchangeRateToLkr`	number	302.5	**Frozen at save time** (ADR-0001); derived from what his bank credited (**FR-INC-6**)
`amountInLkr`	number	13612.5	Stored, so October still matches his bank statement in March (**NFR-6**)
`status`	string	`RECEIVED`	`RECEIVED` or `PENDING_INVOICE`; overdue is derived, never stored (**FR-INC-4**)
`dateReceivedOrExpected`	number		The due date while pending
`isRecurringMonthly`	boolean	false	True for his salary

4.6 Savings goal

Path: `users/{uid}/goals/{goalId}`.

Field	Type	Kavindu example	Why it exists
`title`	string	MacBook Pro M4	
`targetAmountLkr`	number	490000	The target *as he entered it*; never overwritten with the rate-adjusted value, so it cannot be scaled twice
`currentSavedLkr`	number	11200	Never changed by exchange-rate movements
`targetDateMillis`, `createdDateMillis`	number	12 months out	Drives the required pace (**FR-GOAL-2**)
`usdReferencePrice`	number or null	1599	Apple's USD price (**FR-GOAL-6**)
`usdRateAtSetLkr`, `usdRateSetAtMillis`	number or null	300.0	Rate and time when the target was set

With a USD price, the app shows target × today's rate ÷ rate at set. If the rupee weakens from 300 to 306, his target becomes LKR 499,800 and the goal card says "Target + LKR 9,800 since . . . ".

4.7 Multi-currency handling

1. **Everything is reported in LKR, and nothing is recomputed later.** Each income entry stores its original amount, its currency, the rate used and the LKR result. A later rate change never moves last month's numbers, so October still matches his bank statement in March (**NFR-6**).
2. **The fetched rate is only a suggestion** (ADR-0001). The form pre-fills the day's rate from open.er-api.com (USD→LKR) and CoinGecko (USDT, ETH), cached once a day (C-17). When his AdSense payout lands he can type the LKR his bank actually credited, and the effective rate is derived from it (**FR-INC-5**, **FR-INC-6**).
3. **Offline fallback.** If no rate has ever been fetched, the app assumes USD→LKR 305.00, 1 USDT = 1 USD and 1 ETH = 2,600 USD, and labels the rate "Offline benchmark".
4. **Crypto results are signed.** A loss is a negative `originalAmount` and `amountInLkr`; the Income Basis keeps losses and leaves profits out of the three-month average (**FR-INC-7**, **FR-DASH-3**).
5. **The goal is the only value that follows the live rate**, and only when it has a USD reference price (**FR-GOAL-6**).

4.8 Maps on the user document

```
"subscriptionReviews": {
  "netflix_premium": { "decision": "CANCELLED", "decidedAt": 1730700000000, "
    consecutiveUnusedMonths": 0 },
  "gym_membership":  { "decision": "KEEP",      "decidedAt": 1730600000000, "
    consecutiveUnusedMonths": 2 }
},
"seasonalReserves": {
  "vesak_festival_2027": { "amountLkr": 20000, "setAsideAt": 1776000000000, "peakDateMillis":
    1778800000000 }
},
"onboarding": {
  "monthlySalaryLkr": 126400,
  "incomeSources": { "SALARY": "LOW", "FREELANCE": "MODERATE", "ADSENSE": "MODERATE", "CRYPTO":
    "VOLATILE" },
  "fixedCommitments": [ { "title": "Rent", "amountLkr": 34000, "category": "RENT" },
                        { "title": "Fiber Broadband", "amountLkr": 4800, "category": "UTILITIES"
    } ],
  "completedAt": 1730750000000
}
```

- **Subscription keys** are the merchant title lower-cased with non-alphanumerics replaced by underscores. A charge after `CANCELLED` marks the subscription "still charging"; two consecutive "didn't use it" answers make its risk High (**FR-SUB-3**, **FR-SUB-4**).
- **A seasonal reserve is an earmark, not a transaction.** It lowers Safe-to-Spend only in the month it was set aside and until the peak day; after that the festival spending itself is recorded, so keeping the reserve would count the money twice (**FR-LK-5**).
- **Onboarding** records his salary, each active source with its variability and his fixed commitments (**FR-AUTH-4**, **FR-AUTH-5**).

4.9 Configuration: APIT tax bands

Path: `config/apitBands`.

```
{ "effectiveFrom": "2025-04-01",
  "bands": [ { "lowerLkr": 0,      "upperLkr": 100000, "rate": 0.00 },
             { "lowerLkr": 100000, "upperLkr": 141667, "rate": 0.06 },
             { "lowerLkr": 141667, "upperLkr": 183333, "rate": 0.12 },
             { "lowerLkr": 183333, "upperLkr": 225000, "rate": 0.18 },
             { "lowerLkr": 225000, "upperLkr": 266667, "rate": 0.24 },
             { "lowerLkr": 266667, "upperLkr": 308333, "rate": 0.30 },
             { "lowerLkr": 308333, "upperLkr": null,   "rate": 0.36 } ] }
```

The payslip calculator validates the schedule (starts at 0, contiguous, rates between 0 and 1) and falls back to the bands bundled in the app if the document is missing or invalid. For a gross salary of LKR 140,000 this gives APIT of LKR 2,400 and employee EPF of LKR 11,200, leaving a take-home of LKR 126,400 – the figure Kavindu could not recall from his payslip (**P10**).

4.10 Security rules

`firestore.rules` keeps each user to their own records *and* checks what is written, matching the DTOs:

```
match /users/{userId} {
  allow read, write: if isOwner(userId);            // the three maps, merged
  match /expenses/{id}       { allow read, delete: if isOwner(userId);
                               allow create, update: if isOwner(userId) && validExpense(userId);
    }
  match /income_streams/{id} { ... validIncome(userId) }
  match /goals/{id}          { ... validGoal(userId) }
  match /accounts/{id}       { ... validAccount(userId) }
}
match /config/{docId} { allow read: if request.auth != null; allow write: if false; }
```

Record	Checked on every create and update
Expense	`userId` equals the path; title text up to 200 characters; `amountLkr` a number ≥ 0; `category` one of the eight categories; `timestamp` a number
Income	`userId`; `source` one of Salary, Freelance, AdSense, Crypto; `currency` one of LKR, USD, USDT, ETH; `status` one of three; amounts are numbers (negative allowed for a crypto loss); `exchangeRateToLkr` a number > 0
Goal	`userId`; title; target and saved amounts numbers ≥ 0; USD fields a number or null
Account	`userId`; name; `type` one of the four; `balanceLkr` a number; `freezeBreakCount` a whole number ≥ 0; account number at most 34 characters (IBAN length)

The rules check the document as it will be after the write, so partial updates (marking an invoice received, adding to the goal) are validated whole. Unknown extra fields are allowed, so an older app version keeps working. Unauthenticated requests are refused, another signed-in user can never read or write Kavindu's documents, a record cannot claim another user's `userId`, configuration is read-only from the app, and every other path is denied by default. The rules were tested against the Firestore emulator with 21 cases: 10 app-shaped writes, reads and deletes allowed, and 11 invalid or foreign writes denied.

4.11 Indexing strategy

Query	Index that serves it
`expenses` ordered by `timestamp`, newest first (live)	Automatic single-field index on `timestamp`
`income_streams` ordered by `dateReceivedOrExpected` (live)	Automatic single-field index
`goals`, `accounts`: whole collection (live)	None needed: no filter or sort
`accounts limit(1)` from the server (is this a new user?)	None needed
`users/{uid}`, `config/apitBands` by ID	Document lookup, no index

No composite index is needed, so `firestore.indexes.json` declares none. This is deliberate. Filtering happens on the device: the Analytics periods and the category, account and date filters (**FR-ANA-5**), Safe-to-Spend and the subscription scan all run over records already in the Room cache, which must be there anyway for offline use (**NFR-3**). One person's records – hundreds a year – filter instantly; a server query per filter would cost reads, fail offline, and need a composite index for each filter-and-sort pair. If a history ever grew to tens of thousands of records, the expense listener would load only a recent window (`where timestamp >= start`, ordered by `timestamp`), which is still a single field. A composite index would only be needed for a server query that filters on one field and sorts on another, such as subscriptions ordered by date.

4.12 Local cache and entity-relationship model

Room (`AppDatabase`, version 3) mirrors the four collections in the tables `accounts`, `expenses`, `income_streams` and `savings_goals`. It is only a cache: it uses `fallbackToDestructiveMigration()`, and pending offline writes are not lost when it is rebuilt because they are queued in Firestore's own cache. For the same reason it is emptied on sign-out and excluded from Android backups and device transfer; Firestore restores his records when he signs in.

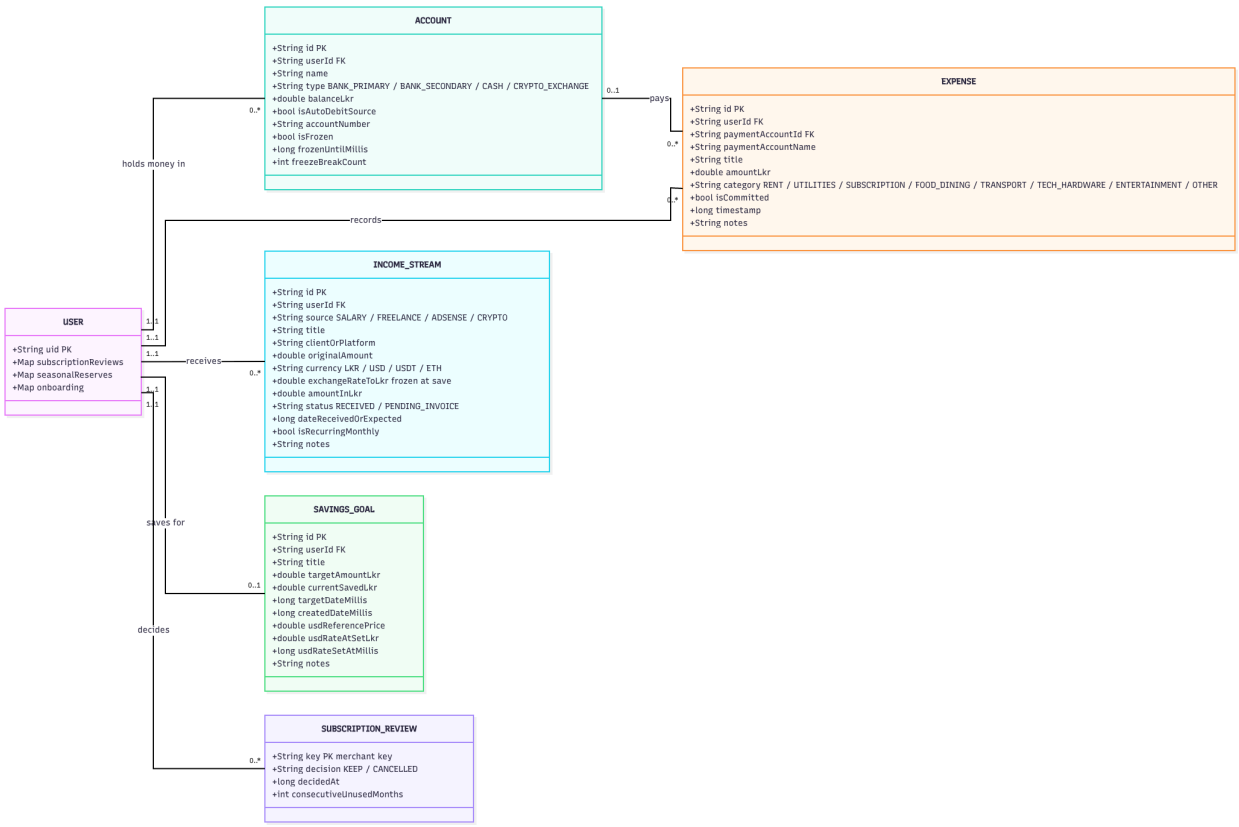


Figure 3: Firestore data model in UML notation. Multiplicities: `1..1` exactly one, `0..1` none or one, `0..*` none or many. Each user has none or many accounts, expenses, income entries and subscription reviews, and at most one savings goal (the app reads the earliest created). An expense is paid from none or one account, because `paymentAccountId` may be empty. The three maps are fields of the user document.

5 Annotated Wireframes

The wireframes below are annotated designs of the main screens, made in Figma: Today, Log an Expense, Income and Quest. Each numbered callout is explained beside the screen: what the element does for Kavindu and which of his problems it answers. The bottom dock (Today, Analytics, Income, Quest, Audit) is the same on every screen after setup. Amounts and names in the wireframes are illustrative values drawn from Kavindu's scenario, not data from a real account.

High-fidelity wireframes (Figma):

https://www.figma.com/design/LvHHjUqY2v2T1Awmhsk4RH/PulseFinance-Annotated-Wireframes?node-id=16-547&p=f

5.1 Today (dashboard)

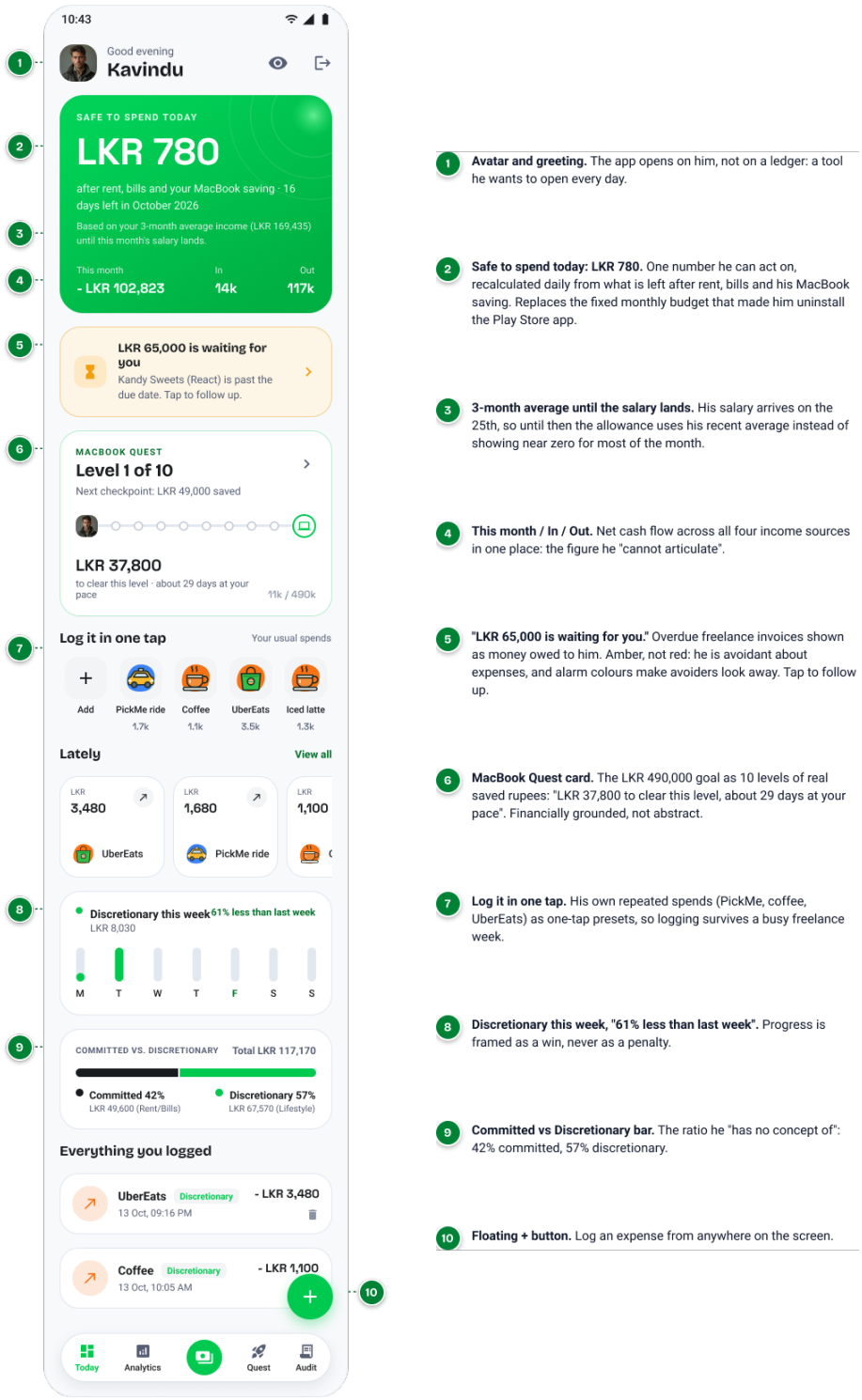


Figure 4: Annotated wireframe of the Today screen, the first screen after sign-in.

5.2 Log an expense

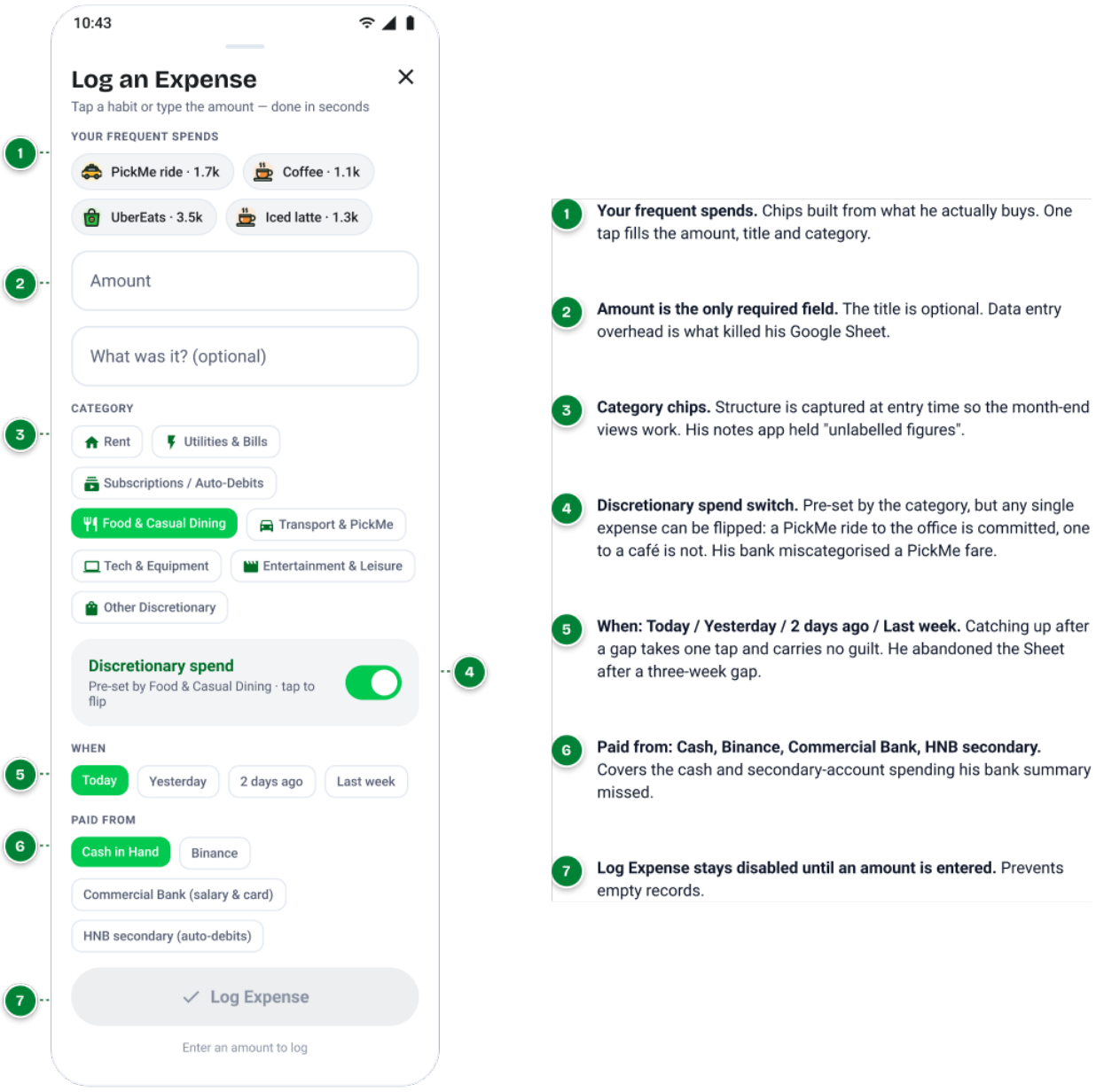


Figure 5: Annotated wireframe of the Log an Expense sheet, opened from a preset, the + button or a shared bank SMS.

5.3 Income hub

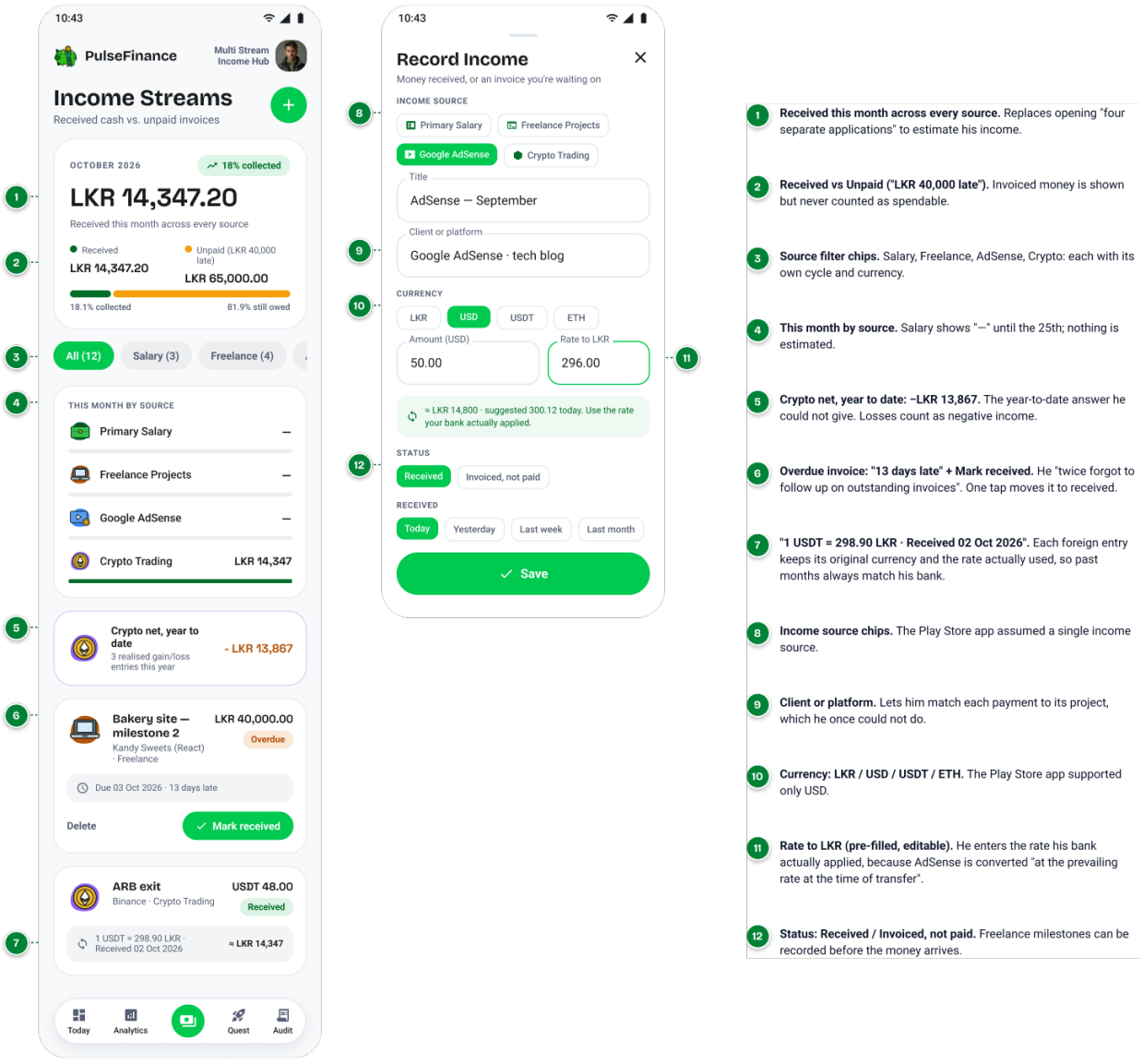


Figure 6: Annotated wireframes of the Income hub (left) and the Record Income sheet (right).

5.4 Quest (savings goal)

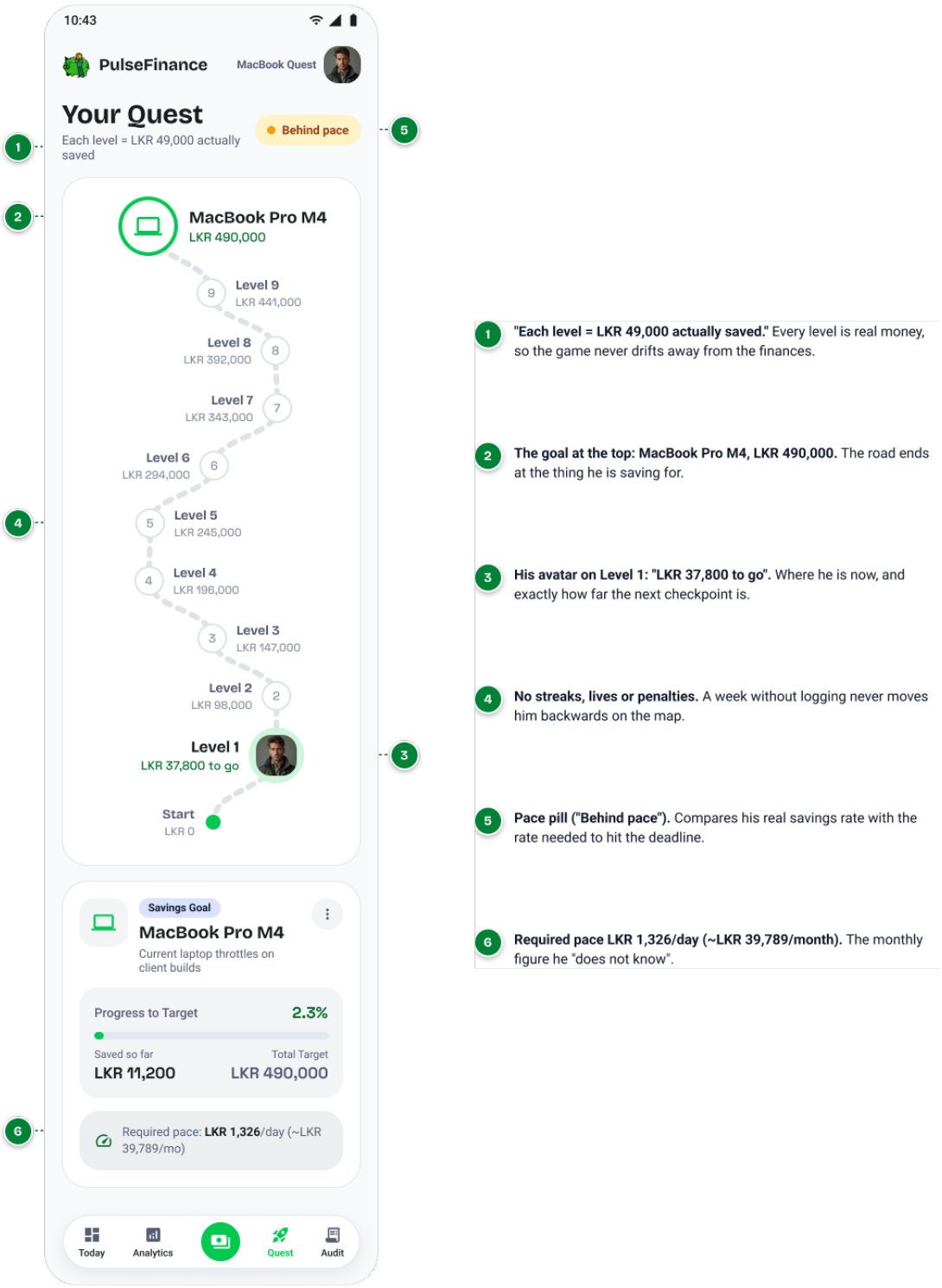


Figure 7: Annotated wireframe of the Quest screen: the savings goal as a road of real-rupee levels.

5.5 Visual language

The theme is light (porcelain white `#F7F9FB`, white cards) with one signal green (`#00C950`, text `#006E28`). Attention states – an overdue invoice, a freeze break, an error – use amber (`#F59E0B`) with deep-amber text (`#B45309`), not red: Kavindu avoids looking at money he is worried about, and alarm colours make avoiders look away. **One deliberate exception:** the subscription audit marks "Still charging" and the Cancel actions in rose (`#E11D48`), because a forgotten subscription draining money is the one thing he should not look away from (**P6**).

- **Material 3 theming.** `Theme.kt` maps the brand palette onto every Material 3 colour role (`primary`, `primaryContainer`, `background`, `surface` and its containers, `outlineVariant`, `error`, ...), and `MaterialTheme.pulseColors` adds the few roles Material 3 lacks (warning amber, the audit's status palettes, chart colours). Screens read colours only through the theme; none hard-codes one.
- **Illustrated icons.** Things he recognises – a coffee, a PickMe, rent, his bank, a salary, his goal – use small illustrated icons in the Storyset style; plain Material icons are kept for controls such as close, the eye toggle and a frozen account's lock. The bottom dock uses the same style, with the selected tab in colour on a soft green blob.
- **One pop-up for feedback.** Every confirmation and error appears as the same white card below the status bar, showing what was saved ("Expense logged · LKR 850 · Coffee", with the cup), then dismisses itself.
- **Motion.** Tabs use a Material shared-axis transition (300 ms) in the direction of the tab tapped.

Money figures use Space Grotesk; headings use Bricolage Grotesque; illustrations are from Storyset. The full design system is in `docs/DESIGN_SYSTEM.md`.

6 Design Decisions and Justifications

Each decision is stated with the part of Kavindu's scenario that forced it, the alternatives considered and the consequences accepted.

DD-1 Offline-first: Room cache in front of Firestore

Context	He records spending at the point of purchase – in a PickMe, in a café – often with poor signal (**NFR-1**, **NFR-3**). A screen that waits for the network is a screen he stops opening (F1).
Alternatives	Firestore only (its cache alone handles reads, but every screen would need its own listener and the UI would depend on SDK caching behaviour); Room only (no sync, the exact failure of the Play Store app, F2).
Consequences	Screens read only from Room and open instantly; Firestore listeners mirror changes while a screen is open (`cacheBackedFlow`); writes go to both, and Firestore's persistent queue uploads them later. Cost: two stores to keep consistent, solved by having the listener `replaceAll` the user's rows in one transaction.

DD-2 The received LKR decides; the exchange rate is frozen per entry (ADR-0001)

Context His AdSense is paid in USD but credited in LKR at his bank's rate, which differs from the market rate. "Never records what he actually receives" (**P3**).

Alternatives Store USD and convert at display time with today's rate (past months would change every day and never match his bank); store only LKR (loses the original amount and currency, **FR-INC-1**).

Consequences The fetched rate pre-fills the form only as a suggestion; he can type the LKR that arrived and the effective rate is derived. Original amount, currency, rate and LKR result are all stored and never recomputed, so October still reconciles with his bank statement in March (**NFR-6**).

DD-3 An Income Basis instead of a monthly budget

Context His salary lands on the 25th, so "income so far" is near zero for most of the month; he is also optimistic about income and impulsive.

Alternatives A fixed monthly budget (what failed in F2); this month's received income only (an allowance of zero until the 25th).

Consequences The allowance uses the higher of this month's realised income and the previous three months' average. Crypto profits are left out of that average but losses are kept (**FR-DASH-3**), so one lucky trade cannot inflate what he is told he can spend.

DD-4 Unpaid invoices are tracked but never spendable; "overdue" is derived

Context He has twice forgotten to follow up freelance milestones (**P2**), and money a client has not paid is not money he has.

Alternatives Count invoices in the allowance (overstates spending power); store an `OVERDUE` status (needs someone or a job to relabel it).

Consequences A pending invoice is stored with its due date; `effectiveStatus()` reports it as overdue once the date passes, and the dashboard banner surfaces it with one-tap "Mark received" (**FR-INC-4**, **FR-DASH-5**).

DD-5 The category proposes committed or discretionary; he decides

Context His bank labelled a PickMe fare "online retail" (F4). He has never seen his committed/discretionary split (**P7**).

Alternatives Fully automatic classification (repeats F4's mistake); a manual flag with no default (adds a decision to every entry).

Consequences Each category has a sensible default (Rent committed, Food & Dining discretionary) that he can flip per expense in one tap (**FR-EXP-2**).

DD-6 Bank SMS through the share sheet, not READ_SMS

Context Card spending is the easiest to forget, and his bank sends an SMS for each transaction (**P5**).

Alternatives Request READ_SMS and read every message (restricted by Google Play, and a large privacy cost for a finance app); no SMS support at all.

Consequences He shares an SMS into PulseFinance; `BankSmsParser` extracts amount, merchant and card digits into a draft he confirms (**FR-EXP-7**, C-15). He stays in control, and the app holds no SMS permission.

DD-7 A Spending Freeze is a self-commitment, and it never blocks

Context He is impulsive with discretionary spending (**P11**), but the app has no bank connection (C-13). A tool that refuses to record reality stops being trusted.

Alternatives No friction at all; refusing to log spending from a frozen account (records would then lie).

Consequences He can freeze an account until a date. Spending from it shows the cost in days of MacBook and offers "Log anyway". The expense is always recorded and counted as a Freeze Break (**FR-ACC-3**–**FR-ACC-5**).

DD-8 Subscriptions are detected from his own charges and one question

Context Two subscriptions he meant to cancel six months ago are still charging his secondary account (**P6**); there is no bank feed (C-12).

Alternatives Manual subscription list (he would have to remember what he forgot); geofenced gym detection (out of scope: battery and privacy).

Consequences Subscription-category expenses are grouped by merchant over 120 days; after each charge he answers "Did you use this this month?"; Keep/Cancel decisions are stored so a later charge is caught; app-usage data adds evidence when granted (**FR-SUB-1**–**FR-SUB-5**).

DD-9 Tax bands are data in Firestore

Context He does not understand his payslip (**P10**), and Sri Lankan APIT bands were revised in 2023 and 2025 (C-16).

Alternatives Bands compiled into the app (a new APK for every budget).

Consequences The payslip calculator reads `config/apitBands`, validates the schedule and falls back to bundled bands offline or if the document is invalid (**FR-LK-2**). The threshold itself is data: a revised schedule changes it without code.

DD-10 A USD-linked goal scales the target he entered

Context The MacBook is bought from an authorised reseller in LKR, but its price follows the dollar.

Alternatives Target = USD price × today's rate (drops the reseller's markup and replaces the number he chose); a fixed LKR target (silently drifts from reality).

Consequences The LKR target he entered is stored with the rate at that moment; the app shows target × $\text{rate}_{\text{today}} \div \text{rate}_{\text{set}}$ and states the change. The stored target is never overwritten, so it cannot be scaled twice (**FR-GOAL-6**).

DD-11 Seasonal Peaks from the calendar and from his own history

Context Avurudu, Vesak, Poson and December spending surprises a goal plan, and an impulsive spender benefits from warning in advance (**P11**).

Alternatives Calendar festivals only (misses his own expensive months); a fixed buffer per festival (ignores his real behaviour).

Consequences 30 days before each festival – and, after 12 months of history, before any month at least 20% above his average – he is warned with last year's excess. Averages count only months that have records. "Set aside" stores an earmark that lowers Safe-to-Spend until the peak, not a transaction (**FR-LK-4**, **FR-LK-5**).

DD-12 Gaps are missing data, never failure

Context A three-week gap made his Google Sheet feel like failure and he abandoned it (F1, **P9**).

Alternatives Streaks and daily targets (common in habit apps, and exactly what punishes a gap).

Consequences No streaks or lives; the goal's quest levels are real rupee amounts, so progress only moves forward; the savings rate and seasonal averages count only periods with data (**NFR-2**).

DD-13 Amber instead of red, and one number first

Context He avoids looking at money he is worried about; his tools became "too unpleasant to open" (F1–F4).

Alternatives A conventional red/amber/green status palette; a dashboard of many equal-weight figures.

Consequences Attention states are amber with dark text; the only red is in the subscription audit, for charges he must not ignore (**P6**). The dashboard leads with one actionable number, Safe to Spend Today, with net cash flow as context underneath.

DD-14 Sensitive numbers behind biometrics; no card data at all

Context He needs his bank account number for freelance invoices (P2), but a finance app must not leave it on screen.

Alternatives Plain display; storing card numbers and CVVs for a "card" view (forbidden, C-14).

Consequences Account numbers are masked (• • • • 1234) and revealed only after `BiometricPrompt`; a copied number is cleared from the clipboard after 60 seconds; screens are excluded from screenshots with `FLAG_SECURE`; no card number or CVV exists anywhere in the app (FR-ACC-6, FR-ACC-7, NFR-10).

DD-15 All calculations in domain use cases with unit tests

Context His Google Sheet's formulas broke when rows were inserted (F1). The numbers are the product: if they are wrong once, he stops trusting them.

Alternatives Calculations inside ViewModels or Composables (untestable on the JVM, and against C-3).

Consequences Safe-to-Spend, goal velocity, savings rate, payslip, subscriptions, seasonal peaks, SMS parsing and CSV export are plain Kotlin use cases with JVM unit tests run in CI on every pull request. Composables hold no rules at all, and the domain layer has no Android imports.

DD-16 One quantified goal

Context The scenario is about one goal, the MacBook, which is "emotionally present and financially invisible" (P8).

Alternatives Several goals competing for the same savings (needs allocation rules the scenario does not ask for).

Consequences The model can store more goals, but the app reads and paces one – the earliest created – so every screen speaks about the same target.

7 Implementation Status, Testing and Delivery

7.1 Requirement status

Status	Count	Requirements
Done	68	All requirements not listed below
Partial	3	FR-SUB-5 and NFR-12 (usage access is not yet requested in context; app-usage risk works once access is granted in Settings); NFR-2 (gap tolerance verified for the savings rate and seasonal averages, not yet for every view)
Planned	2	FR-AUTH-6 (emergency-runway buffer); NFR-8 (accessibility at 200% font scale and colour-independent status not yet verified)

7.2 Testing

Domain logic is covered by JVM unit tests in `app/src/test`, run with `./gradlew testDebugUnitTest`: the Safe-to-Spend summary, goal velocity and the USD-linked target, savings rate, the payslip and APIT bands including the Firestore fallback, subscription detection, Seasonal Peaks including personal peaks and gap-aware averages, the bank SMS parser and CSV export. Representative cases are taken from Kavindu's scenario – for example, a LKR 140,000 gross salary must produce APIT of LKR 2,400, and a goal of LKR 490,000 with LKR 11,200 saved over 360 days must require LKR 1,330 per day. Edge cases have their own tests: a three-week logging gap must not count as a month of zero saving, a crypto loss larger than a month's salary must stay in the Income Basis, and an invoice's days overdue and the grouping of subscriptions by account are tested in the domain. In total there are 71 unit tests.

The Firestore security rules were tested separately against the Firestore emulator: 21 cases, of which 10 app-shaped writes, reads and deletes (including partial updates and a negative crypto amount) are allowed and 11 invalid or foreign writes are denied. Visual changes are checked on an emulator and on a Galaxy S23.

7.3 Project management

The project was run on GitHub from the requirements onwards, so every requirement can be followed from the SRS to the code that implements it and the pull request that merged it.

- **One issue per requirement.** Each functional and non-functional requirement in Section 2 became a user-story issue with acceptance criteria, labelled with its MoSCoW priority (`priority: must`, `should`, `could`). Bugs and engineering tasks found along the way became their own issues.
- **Epics with sub-issues.** The issues are grouped under eleven epics – one per feature area, such as Sign-in & onboarding, Multi-stream income, Subscription audit and Quality – using GitHub sub-issues, so each epic shows its progress (for example 15 of 18 done).
- **A project board.** The *PulseFinance – SE3092* board tracks every issue with its assignee, status (Todo, In Progress, Done), work type, priority and linked pull requests (Figure 8). Built-in workflows add new issues to the board and move them as their pull requests merge.
- **Pull requests close issues.** Work happens on a branch per issue; its pull request says `Closes #N`, describes the change with evidence, and states its risk ("Merge Danger"). Merging closes the issue and marks it Done on the board (Figure 9).
- **Checks on every pull request.** Three GitHub Actions workflows run: an Android quality gate (unit tests, lint and a debug build), the Firestore rules tests, and the release build.

Item	Total	Done	Notes
Epics	11	–	One per feature area; parents of the issues below
User stories	82	78	One per requirement, with acceptance criteria and priority
Bugs	9	8	Found in reviews and audits
Tasks	15	10	Engineering, delivery and documentation work
Pull requests	38	32 merged	23 close issues with `Closes #N`; Hesara 18, Sandali 15, Shazaan 5

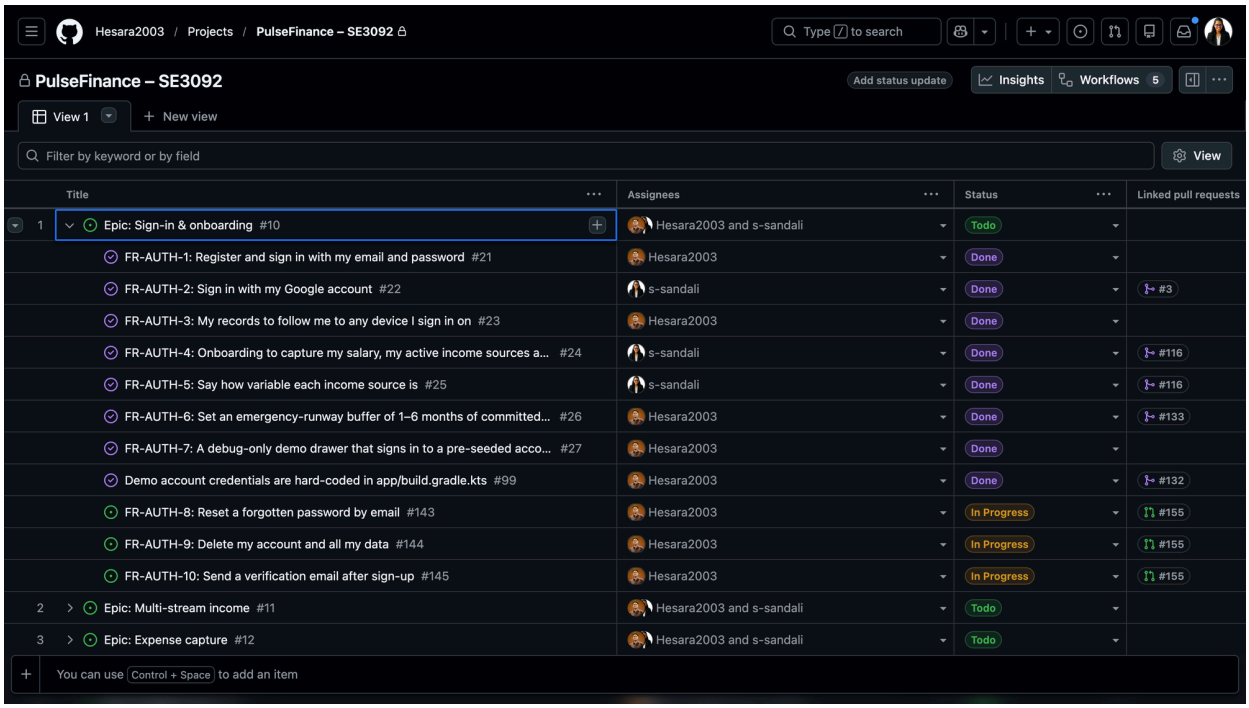


Figure 8: The project board: the Sign-in & onboarding epic expanded, each requirement with its assignee, status and linked pull request.

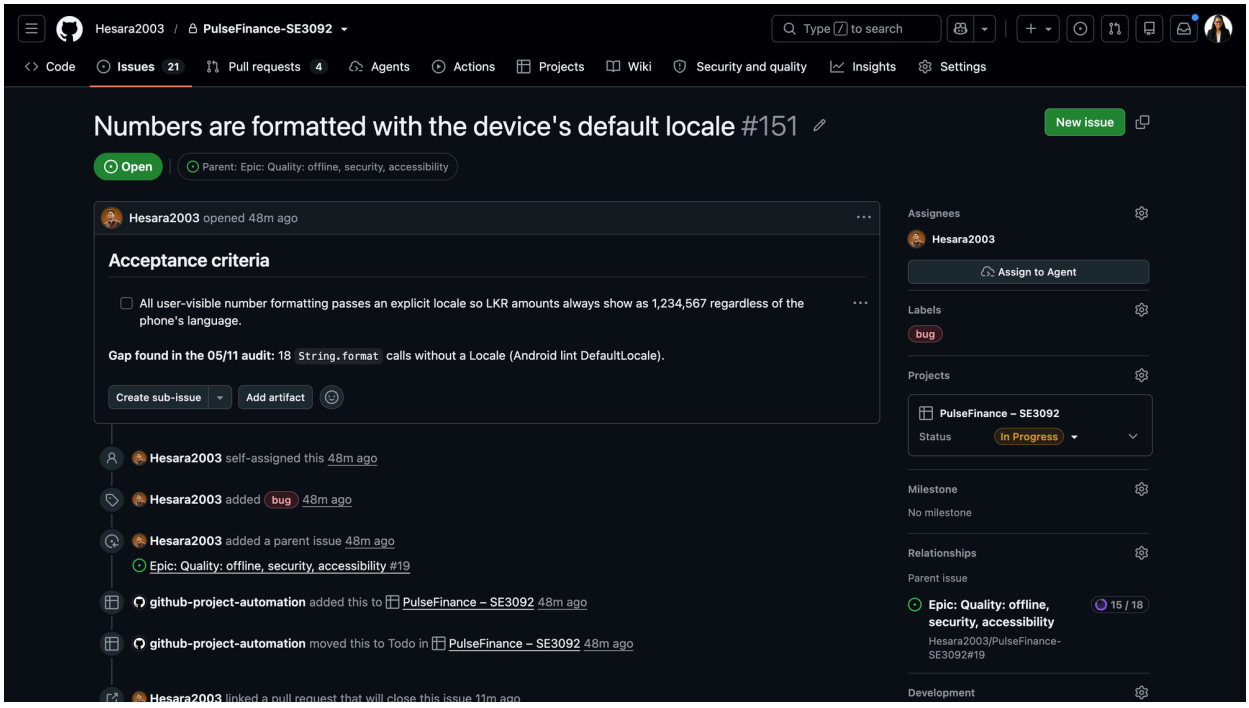


Figure 9: An issue with acceptance criteria, a label, its parent epic (15 of 18 sub-issues done), its board status and a linked pull request.

7.4 Delivery

Source code (GitHub):

https://github.com/Hesara2003/PulseFinance-SE3092

Latest installable build (APK, Google Drive):

https://drive.google.com/drive/folders/1GRAAyZV_luvfbuwS06Pa01GojCZQkg7d

- **Repository and workflow.** Every change goes through a pull request on `Hesara2003/PulseFinance-SE3092`, is reviewed by another team member and is merged with a merge commit so that the history records who did what.
- **Continuous integration.** `ci.yml` runs the unit tests, lint and a debug build on pull requests and pushes to `main`.
- **Release.** `release-apk-to-drive.yml` builds and signs the release APK and AAB on every push to `main` and uploads them to the team's Google Drive folder, so an installable build of the current `main` is always available.

7.5 Known limitations

- The Room cache is protected by the app sandbox but not encrypted.
- Accounts are the four created on first sign-in; they cannot yet be renamed, added or removed, and an account number cannot be entered in the app.
- Google sign-in in release builds needs the release key's SHA-1 registered in Firebase.

7.6 Team contributions

The following summary is taken from the Git history of the repository.

Member	Main contributions
Hesara	Initial architecture, Room and repository layer, core financial use cases, CI/CD and release pipeline; payslip and APIT, bank SMS parsing, CSV export, Seasonal Peaks, biometric reveal, Spending Freeze, analytics periods and donut (24/10); APIT bands from Firestore, personal Seasonal Peaks and the USD-linked goal (04/11); repository sync and documentation; this technical report.
Sandali	Google sign-in, cache-backed Firestore sync, subscription detection and review, savings rate, formatters and transaction visuals (13/10); SRS, ADR-0001 and glossary, light-green redesign and the goal quest (16/10); first-run setup, analytics account and date filters, Seasonal Peak "set aside", `FLAG_SECURE`, illustrated icons, release signing fix and UI polish (05/11); validated Firestore rules and the indexes file, sign-out cache clearing, backup exclusion, demo credentials out of the repo, the gap-aware savings rate and Income Basis fixes, Material 3 theming, splitting the Today and Audit screens, and documentation updates (05/11).
Shazaan	QA unit tests for goal pacing, opportunity cost and currency conversion (17/10, being ported to the current API); assigned gap tolerance (NFR-2), accessibility (NFR-8), app-usage risk and in-context permissions (FR-SUB-5, NFR-12).

8 Conclusion

Kavindu's problem was never a lack of discipline; it was that every tool he tried misrepresented his financial life – one income, one currency, one card, fragile formulas – until he stopped trusting it. PulseFinance answers each of those failures with a specific design decision: income recorded

at what actually arrived and frozen there, unpaid invoices kept out of what he can spend, every channel his money leaves through in scope, subscriptions surfaced from his own charges, and a goal expressed as a daily pace compared with what he really saves. The architecture – an offline-first MVVM app over a user-scoped Firestore schema – exists to keep those numbers fast, private and trustworthy enough that he keeps opening the app. Sixty-eight of seventy-three requirements are implemented; the remaining five are known, small and tracked.